面向访存与部分归约的 GPU 张量布局选择

以 Triton 为切入点：问题表征、现有编译器机制与代价建模方案

科研进展、现有选择机制与后续研究计划

Chen Miaoming

2026 年 10 月 15 日

计算语义 → 布局与现有选择机制 → 受控实验 → 代价建模计划

切入点：取数布局与部分归约的局部性

从 TMA 的版本回归出发，研究哪些配置需要改选布局，哪些配置应保留默认。

- 编译器需要把一个逻辑张量分配到寄存器、lane 和 warp；同一数学计算可以有不同布局。
- 连续 N 维的向量化可能降低取数成本；沿 M 维规约的线程局部性可能减少通信。
- 已有 BF16/max 对照表明：某些配置有明显优化空间，另一些配置的默认布局已接近较快候选。

研究问题

- 如何用编译期信息和硬件校准参数预测完整“取数—归约—输出”的成本?
- 能否在未见形状上选到较好布局，并在收益小或预测不确定时回退默认?

[1] Issue #9703；[2,3] H100 与 A10 既有实验

代表性 Triton 程序：TMA 后逐列 max

输入 BF16 [1,8192,8192]；加载 [1,BM,BN]，沿 M 维（axis=1）规约，输出 [1,BN]。

```
import triton
import triton.language as tl
@triton.jit
def partial_max(x_desc, out, BM: tl.constexpr, BN: tl.constexpr):
    pid = tl.program_id(0)
    pm, pn = pid // (8192 // BN), pid % (8192 // BN)
    x = x_desc.load([0, pm * BM, pn * BN])  # [1, BM, BN]
    x = x.to(tl.float32)
    y = tl.max(x, axis=1)                   # [1, BN]
    cols = pn * BN + tl.arange(0, BN)
    offs = pm * 8192 + cols
    tl.store(out + offs, tl.reshape(y, [BN]))
```

本页展示 TMA 路径；x_desc 在 host 端构造，descriptor 构造不在 kernel 计时区间。

原始几何：32×128 / 4 warps；补充几何：32×64 / 8 warps。普通 tl.load 完整代码见附录 A。

[1] 原始 Issue #9703；host-created descriptor，BF16 partial max

计算模式：逐列归约，消去 M 维、保留 N 维

用 4×4 矩阵说明一个 tile 内的计算；全局 M 维最大值还需要合并不同 tile 的部分结果。

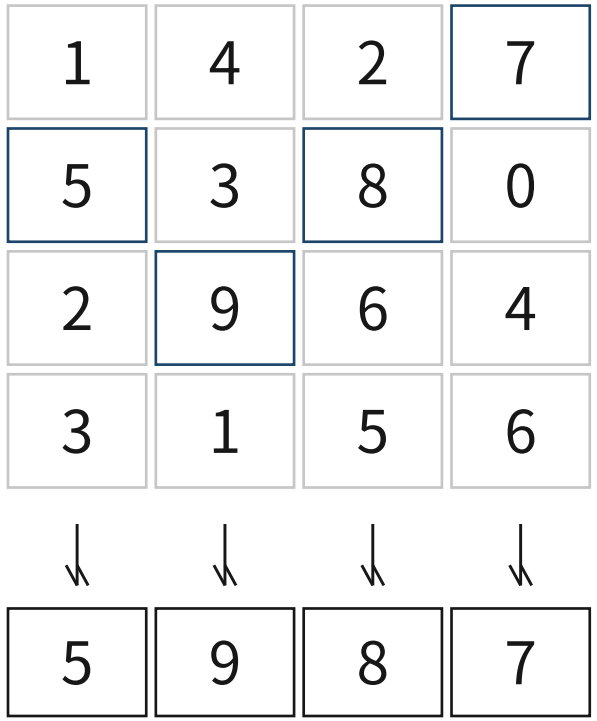


图 1. 每列产生一个输出

$$y[n] = \max\{x[m,n] : 0 \leqslant m < BM\}$$

- 逐列取 max：消去行索引 M，保留列索引 N。
- 每个 CTA 对自己的 tile 计算局部结果。
- 一列的 BM 个值必须合并为一个值。
- 不同布局改变这些值的线程归属，数学目标保持相同。

[1] Descriptor-load / max reproducer

两种取数路径，归约算术都使用寄存器中的值

TMA 的初始 shared staging，与归约/输出转换使用的 shared 临时存储，需要分开。

TMA / host descriptor



普通 tl.load



- 跨线程规约或结果重排仍可能使用 shared memory 和同步；它们不是普通 load 的初始取数。
- vec = sizePerThread[N] 描述基本连续元素归属；不是 TMA 传输宽度，也不等于硬件加载次数。

[1,4] Triton/Gluon 数据路径；[3] A10 归档 SASS 与 NCU

三种输入布局：参数、重复覆盖与同列贡献线程

输入 tile=32×128；均为 1 CTA、4 warp、128 线程。参数按 [M,N] 书写，N 优先。

参数 / 归属	TMA 3.5.1（报告）	TMA 3.6.0（报告）	早期 vec2 原型
sizePerThread	[1,1]	[1,8]	[1,2]
threadsPerWarp	[1,32]	[2,16]	[1,32]
warpsPerCTA	[1,4]	[4,1]	[2,2]
CTA 基本覆盖（逐维相乘）	1×128	8×128	2×128
沿 M 重复次数（32 / 覆盖行数）	32	4	16
每线程元素分布	32 行 × 1 列	4 行 × 8 列	16 行 × 2 列
每线程总元素数	32	32	32
同列贡献线程数	1 lane × 1 warp = 1	2 lanes × 4 warps = 8	1 lane × 2 warps = 2
同列贡献线程跨几个 warp	1	4	2

sizePerThread：每线程的基本元素小块；threadsPerWarp：warp 内 lane 如何分配。

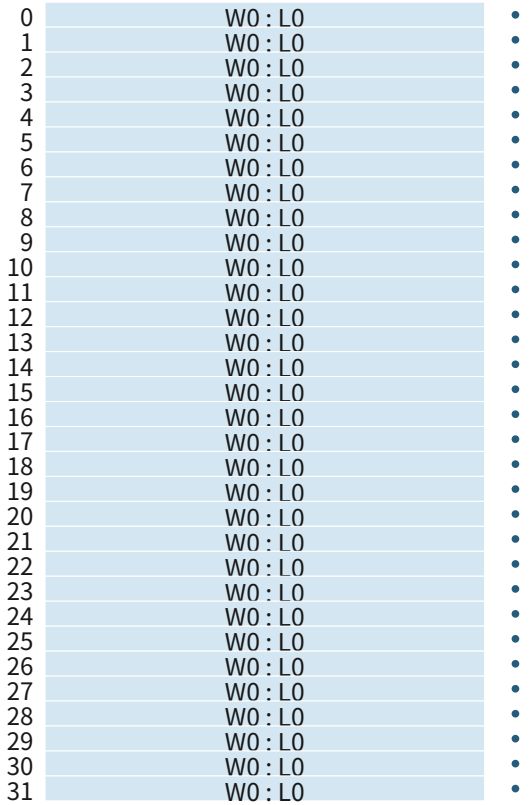
warpsPerCTA：CTA 内 warp 如何分配。基本覆盖 = 三个参数逐维相乘。

重复覆盖不新增线程。第三列仅作布局示例；早期选择规则已弃用，新模型尚待建立。

第 0 列的 32 个值分别归哪个线程?

TMA 3.5.1（原报告）

基本覆盖 1×128；沿 M 重复 32 次

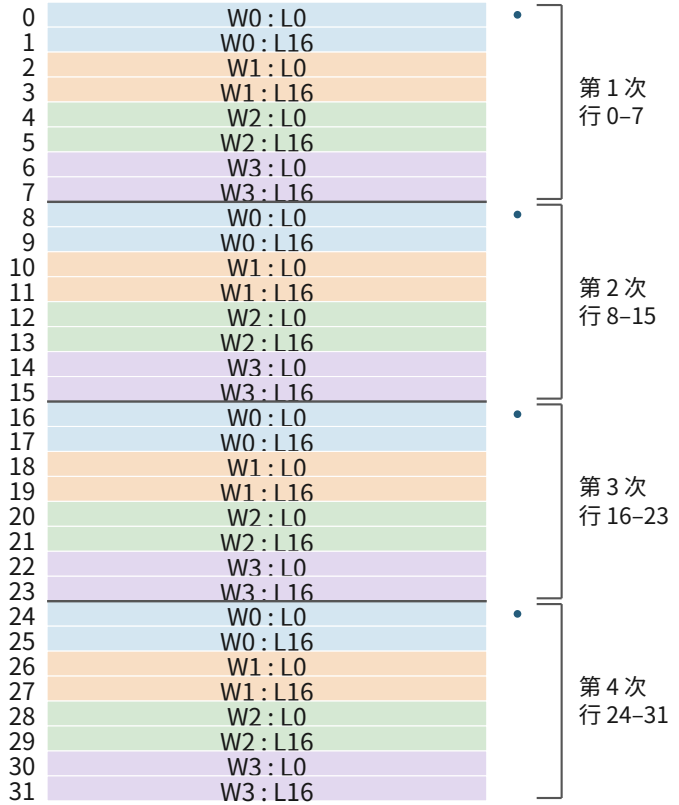


同列：1 个线程，跨 1 个 warp

每线程持有该列 32 个值

TMA 3.6.0 / 固定 baseline

基本覆盖 8×128；沿 M 重复 4 次

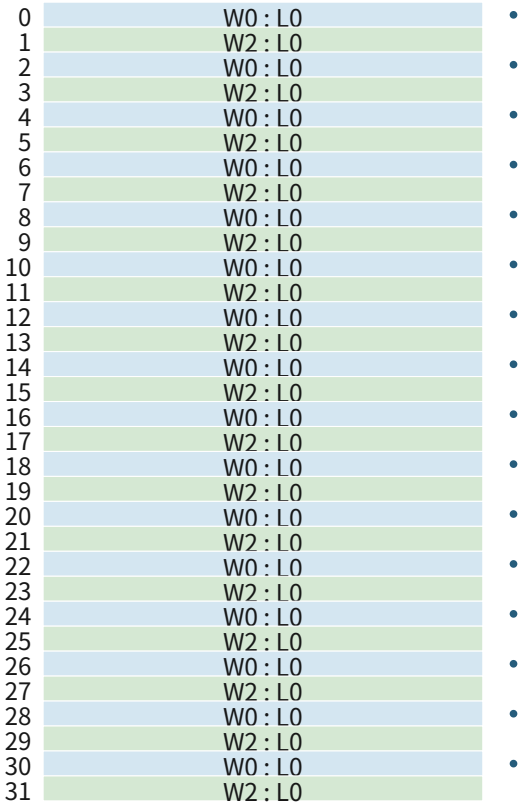


同列：8 个线程，跨 4 个 warp

每线程持有该列 4 个值

早期 vec2 原型：布局示例

基本覆盖 2×128；沿 M 重复 16 次



同列：2 个线程，跨 2 个 warp

每线程持有该列 16 个值

Warp 0　Warp 1　Warp 2　Warp 3　圆点标出同一个线程 W0:L0 的全部持有位置。

仅表示输入归属；当前 baseline 输入编码同 3.6.0，lowering 可不同。W=warp 编号；L=lane 编号。

TMA 与普通 load：两个入口，都是先构造访存布局

核对实验 upstream 20d9d98、v3.7.1 与今日 main 4eda6cc；核心选择规则一致，非新增性能实验。

TMA / descriptor

```
# pickDescriptorLoadStoreLayout
E = max(numElems / (warps * 32), 1)
V = min(E, 128 / element_bits)
SPT = [1, ..., V]
```

- N 优先；BF16 的目标上限为 8，FP32 为 4。
- 再按 shape 和 warp 数分配 lane/warp。
- 此函数不读取后续 reduce axis。

普通 tl.load

```
# buildCoalescedEncoding
order <- pointer contiguity
V0 <- min(alignment, contiguity,
          128 / element_bits)
```

- 用指针连续性与对齐约束向量宽度。
- 协调相关同 shape/order 的内存操作，并受每线程元素量限制。
- 再调用同一个 blocked-layout 构造器。

这两个入口没有对本例的 vec1/2/4/8 做完整“加载—部分规约—输出”的校准时间比较。

同样选 vec8，为什么同列贡献线程会从 8 变成 32?

BlockedEncodingAttr 的构造器从连续 N 维开始分配 lane/warp，再把剩余资源放到 M 维。

分配过程	32×128 / 4 warps	32×64 / 8 warps
平均元素 / 线程	4096 / 128 = 32	2048 / 256 = 8
默认 V（BF16）	min(32, 8) = 8	min(8, 8) = 8
N 上的线程位置	128 / 8 = 16	64 / 8 = 8
threadsPerWarp [M,N]	[2,16]	[4,8]
warpsPerCTA [M,N]	[4,1]	[8,1]
每列贡献线程	2 × 4 = 8	4 × 8 = 32

规则使用了 shape 和 warp 数，但没有用部分规约的预测时间来决定如何分配这些线程。

[S3] TritonGPUAttrDefs.td：shape-based builder；[3] 实际 TTGIR

后续 pass 为什么没有把本例改回线程局部布局?

编译器有布局传播与局部优化；本例没有命中能改写它的特定规约模式。

Coalesce → RemoveLayoutConversions → OptimizeThreadLocality

相关阶段	本例中的行为 / 限制
布局传播	descriptor 是 anchor；普通 load 在本例也作为内存 anchor，布局可向 cast / reduce 等传播。
规约模式优化	循环规约变换要求 axis == rank−1，并匹配特定 scf.for 累加结构等条件。
本例	[1,BM,BN] 的 axis=1，不是最后一维；也没有对应循环结构，因此不匹配。

最终 TTGIR 保留 vec8；这说明规则链的结果，不说明编译器比较过所有候选的完整成本。

[S4,S5] RemoveLayoutConversions.cpp / OptimizeThreadLocality.cpp

需要比较完整路径，同时满足资源约束

同列贡献线程少、vec 更小、静态指令更少，都不是完整 kernel 更快的充分条件。

成本 / 约束	布局改变的内容	建模要求
取数路径	TMA shared-load；普通 global/cache load	按实际访问模板校准
线程内规约	每线程同列值数、packed max / widening	预测实际操作及依赖
跨线程合并	lane / warp 归属、shuffle / shared / barrier	校准通信与同步子图
输出路径	结果 SliceLayout、转换与 store	保留实际消费者要求
资源可行性	寄存器、SMEM、spill 与可达驻留	先合法性门，再预测时间

目标是预测成本与适用条件；用微基准校准参数，再用完整 kernel 检验组合误差。

A10 普通 load：默认布局与固定候选的对照

BF16/max；同批 Triton 3.7.1；单位 μs。普通 Triton 与显式布局 Gluon 分别保留产物。

tile / warps	Triton 默认	Gluon vec1	vec2	vec4	vec8
32×128 / 4	298.368	313.264	297.984	296.960	298.880
32×64 / 8	422.400	279.440	285.696	368.640	421.888

- 32×128 / w4：默认实现接近已测较快候选；直接改成 vec1 反而更慢。
- 32×64 / w8：同一 Gluon kernel 的 vec8→vec1，耗时减少约 33.8%（约 1.51× 加速）。

描述性对照，不宣称全局或唯一最优；Triton↔Gluon 的时间差不直接作为纯布局因果收益。

[3] results/local_a10/20261015_gpu0_r2：3 次进程重复，完整输出检查通过

H100 TMA：收益同样依赖形状与 warp 数

固定未修改 upstream 20d9d98；BF16/max；原始 host descriptor 路径；full grid。

tile / warps	vec1 µs	vec2 µs	vec4 µs	vec8 µs
32×128 / 4	56.048	56.160	56.464	57.296
32×64 / 8	61.392	62.048	68.528	112.096

配对 vec1 / vec8 对照	耗时减少估计	95% 同条件同时区间
32×128 / 4	2.332%	[0.801%, 3.841%]
32×64 / 8	46.652%	[41.056%, 51.718%]

原始几何差异小；补充几何有明确机会，vec1/vec2 尚未分出超过 3% 的稳定差异。

[2] Phase 11 Stage B interpretation.md / summary.md：7 次 worker 调用

早期模型的不足：逻辑分数不能替代硬件成本

以下是早期 H100 32×128/w4 归档代码的静态对照，不是本轮测得的阶段时间。

项目	默认 vec8	早期 vec2 候选
旧加载分数（无量纲，已弃用）	4	16
初始加载 PTX 静态条数	4	4
PTX opcode	ld.shared.v4.b32	ldmatrix...x4...b16
BF16→FP32 转换静态条数	8	2
局部操作顺序	BF16 packed max 后转换	BF16 packed max 后转换
初始加载硬件成本	此对照未单独确认	此对照未单独确认

静态结构需要匹配实际 lowering，再经微基准校准；PTX 条数不能换算 requests、wavefront 或时间。

[5] Phase 10 归档 PTX；[6] NCU 指标定义

先验证微基准能测到什么，再校准子图有效成本

首版固定 H100、host TMA、BF16 partial max；循环、反馈和 checksum 的附加工作显式保留。

子图	合法 Gluon 构造	用途
ML 取数	TMA 预填 shared；指定布局读取并形成 checksum	访问模板、有效延迟/吞吐
MR 规约	驻留寄存器；max 与有界、非幂等的数据依赖反馈	局部计算与交换/同步子图
MO 输出	实际结果布局 → 固定输出布局 → store	消费者与转换路径成本
MC 全图	原始 host TMA → load → max → 输出	检验组合预测与资源匹配

- 同一 CUBIN 用 runtime K=1/16/64/256；NCU 核对目标操作确实随 K 增长。
- C1 最多 16 个程序，先检查重复、模板与资源；无法辨识时修订方案，不冒充裸指令延迟。

[7] Phase 11 FOLLOWUP_PLAN.md：review draft，尚未执行 C1

模型在编译期做选择，完整 kernel 的实测用于评价

灰盒：静态依赖与资源结构 + 每架构校准参数；不是逐个运行待优化 kernel 再挑赢家。

编译期特征 / target
shape / dtype / layout
→
依赖图与资源计划
硬件校准参数 θ
→
预测各合法候选时间
不确定则回退默认

- 预测时只用静态信息与已冻结参数；测试 kernel 的实测时间、NCU 与最终寄存器数不作输入。
- 对照默认布局、固定 vec 与简单局部性基线；报告收益/退化、回退覆盖率与最坏选择损失。
- 模型冻结后测新的留出形状；跨架构区分参数直接迁移与仅重校准硬件参数的迁移。

```
selection regret = T(selected) / min(T(same candidate set)) - 1
```

已完成问题表征；下一步是校准可行性与选择模型

把已经完成的证据、尚待验证的机制，以及后续交付分开汇报。

阶段	具体内容 / 可交付	状态
问题表征	TMA/Gluon 与 A10 对照、默认选择机制、原始证据	已完成；分项归因待校准
C1 可行性	8 布局的 ML/MR，检查动态重复、模板与资源	计划中；最多 16 个程序
C2 / C3	正式校准、静态计划、参数辨识与冻结选择器	C1 通过后推进
D1 / D2	新形状留出评价；H100 后再做 B200 等迁移	模型冻结后执行

预期产出：可解释的布局选择模型、可复现实验与编译器原型；上游改动以验证结果为依据。

A10 上运行的普通 Triton DSL

与前述 TMA 程序具有相同数学操作；仍保留 [1,BM,BN]，所以规约 axis=1。

```
@triton.jit
def canonical(x, out, BM: tl.constexpr, BN: tl.constexpr):
    pid = tl.program_id(0)
    pm, pn = pid // (8192 // BN), pid % (8192 // BN)
    rows = pm * BM + tl.arange(0, BM)
    cols = pn * BN + tl.arange(0, BN)
    offsets = tl.expand_dims(rows[:, None] * 8192 + cols[None, :], 0)
    values = tl.load(x + offsets).to(tl.float32)
    result = tl.max(values, axis=1)
    tl.store(out + pm * 8192 + cols, tl.reshape(result, [BN]))
```

输入 [1,8192,8192] BF16；输出 [256,8192] FP32；未在源码指定布局，由编译器自动选择。

[3] local_a10_versions/kernels.py：与上一轮 canonical 函数 AST 一致

A10：没有版本回归，不代表默认布局总是合适

独立版本对照批次；同 Python/Torch/ptxas；统一加载归档 CUBIN 配对计时。

配置	3.5.1 µs	3.6.0 µs	3.7.1 µs
32×128 / 4 warps	298.032	298.224	298.832
32×64 / 8 warps	436.224	436.208	436.976

- 这两个配置中，三个版本的有效 M/N 默认分布均为 vec8，未观察到该发布边界的回归。
- 本表与第 12 页是不同批次，不能把本表约 436 µs 和另一批约 279 µs 混算收益。
- 原始 TMA issue 的版本回归与普通路径的长期优化机会，是两种不同证据。

[3] local_a10_versions/20261015_gpu0_r2：7 次进程重复，2,520 个 event 样本

源码版本、原始证据与后续方案索引

源码与数据分别固定版本；所有新增结论均可回到归档文件核对。

[1] 原始回归	Triton Issue #9703：GB300，TMA descriptor load + partial max
[2] H100 布局实验	results/phase11/stage_b/{summary,interpretation}.md；upstream 20d9d98
[3] A10 两轮实验	results/local_a10/ 与 results/local_a10_versions/；本地 commit 942d380
[4] 布局语义	Triton / Gluon layouts：逻辑元素到 register / lane / warp 的映射
[5] 早期代码审计	Phase10：32×128/w4 BF16 host 路径 baseline / patch 的原始 PTX
[6] NCU 定义	Instructions、Requests、Wavefronts、Bank Conflicts 分别核对
[7] 后续 review draft	phase11/FOLLOWUP_PLAN.md：C1/C2/C3、保留评价与架构迁移
[S1–S5] 当前机制	main 4eda6cc91e86：Coalesce、builder、传播与 ThreadLocality